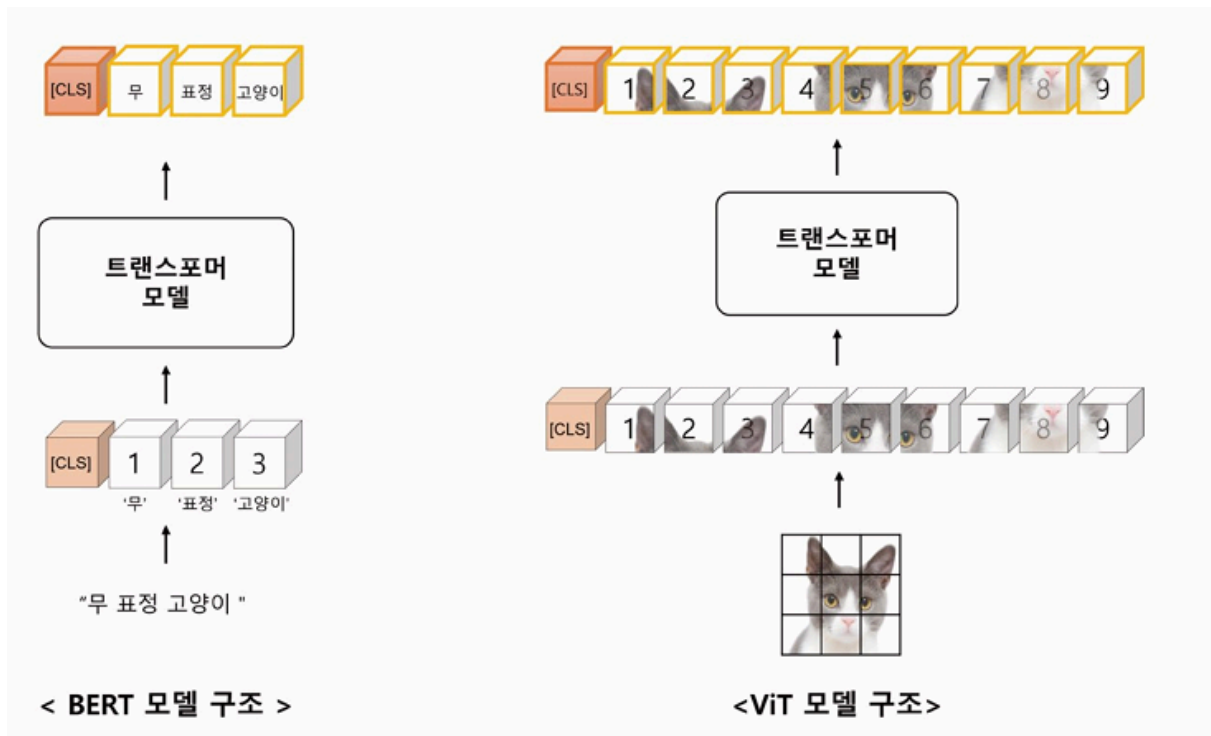


10장_비전 트랜스포머

비전 트랜스포머 개요

- 2020년 구글 *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale* 논문을 통해 소개된 이미지 인식을 위한 딥러닝 모델
- 지역 특징을 추출해 이미지를 분류하는 CNN 합성곱 계층 방법이 아닌 트랜스포머 모델의 셀프 어텐션을 적용해 전체 이미지를 한 번에 처리하는 방식으로 구현
- 이미지 패치가 순차적으로 입력되어 2차원 구조의 이미지 특징을 온전히 반영하지는 X
→ **Swin Transformer, CvT(Convolutional Vision Transformer)**: 물체의 크기나 해상도를 계층적으로 학습
 - Swin Transformer: local window를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 **local window 내부 어텐션, local window 간 어텐션**으로 다양하게 구성해 이미지의 특징 학습. 어텐션 함수에는 상대적 위치 편향을 반영해 어텐션 값 자체에 위치적 정보를 포함함.
 - CvT: 합성곱 연산 과정 + ViT
: 저수준 특징(Low-level Feature)과 고수준 특징(High-level Feature)을 계층적으로 반영.
 - 저수준 특징-작은 단위 / 고수준 특징-큰 단위(전체)
 - 어텐션 연산 과정에서 쿼리(Q), 키(K), 값(V) 중 키와 값을 기존 피쳐 벡터보다 축소해 계산 복잡도 감소

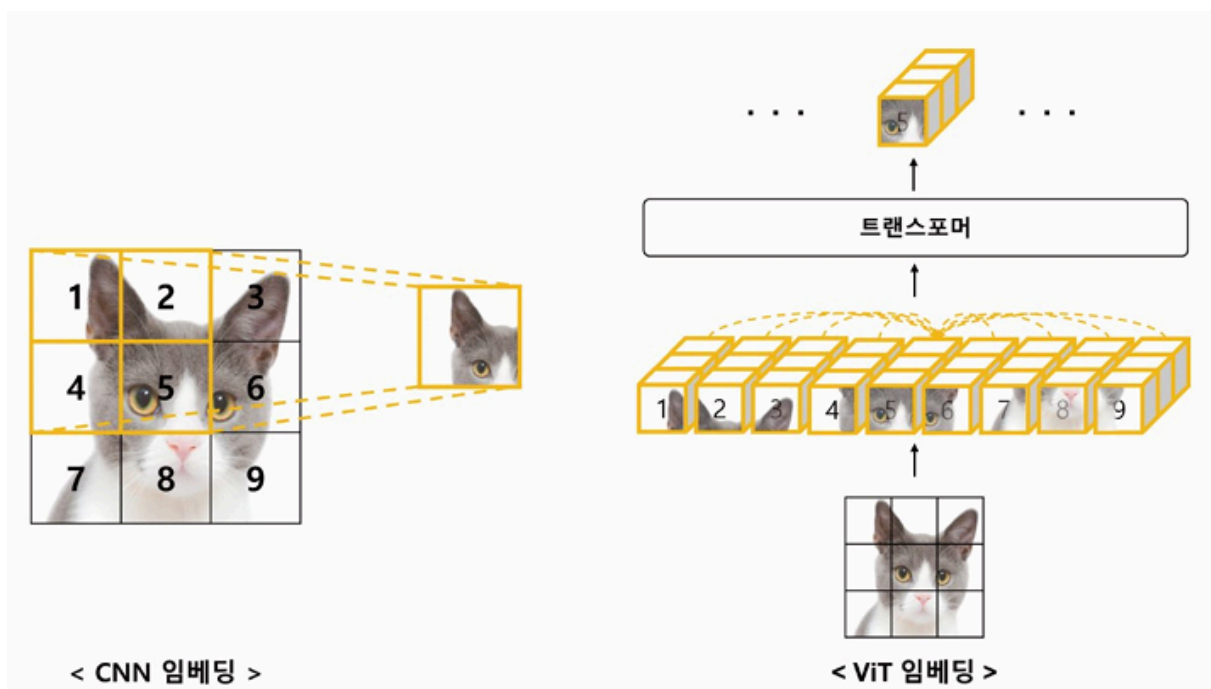
ViT (Vision Transformer)



자연어 처리에 널리 사용되는 BERT 모델과 ViT 모델의 차이

- BERT: 문장보다 작은 단위의 토큰들이 순차적으로 입력됨
- ViT: 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨 → 왼쪽에서 오른쪽, 위에서 아래를 표현되는 sequential 배열

합성곱 모델과 ViT 모델 비교



- **합성곱 신경망**: 이미지 패치 중 일부만 선택해 학습 → 일부를 통해 이미지 전체의 특징 추출
 - 고양이의 오른쪽 눈 부분을 표현하는 데 1, 2, 4, 5만 학습에 관여함
 - 전체 이미지 정보를 표현할 때, 좁은 수용 영역을 가진 합성곱 신경망은 수많은 계층을 필요로 함. 쪼개서 학습하다 보니까 더 많이 쪼개면 학습을 더 많이 시켜야 전체 학습을 완성할 수 있음.
 - 픽셀 단위 이미지 처리
 - 크기가 다양한 이미지를 모두 동일한 방식으로 처리 가능
 - 이미지의 공간적인 위치 정보 고려 → 변환도 가능
- **ViT**: 이미지를 작은 패치들로 나눠 **각 패치 간의 상관관계** 학습. 셀프 어텐션 방법을 사용해 모든 이미지 패치가 서로에게 주는 영향을 고려해 이미지의 전체 특징을 추출함. → 모든 이미지 패치가 학습에 관여
 - 고양이 이미지의 모든 패치가 모든 학습에 관여
 - **어텐션 거리(Attention Distance)**를 계산해 하나의 ViT 레이어로 전체 이미지 정보 쉽게 표현
 - 어텐션 거리: 쿼리 벡터와 값 벡터 사이의 유사도를 내적으로 계산한 것
 - 패치 단위 이미지 처리 → 더 작은 모델로 높은 성능을 얻을 수 있음
 - 입력 이미지의 크기 고정 → 크기 다른 이미지를 처리하려면 이미지 크기 맞추는 전처리를 해야 함
 - 공간적 위치 정보 X, 패치 간의 상대적인 위치 정보만 고려 → 이미지 변환(transformation)에 취약

ViT의 귀납적 편향

귀납적 편향(Inductive Bias): 일반화 성능 향상을 위한 모델의 가정(assumption)

- ex) 이미지 데이터: 공간적 관계를 잘 표현 → **지역적(local)** 편향을 가진 합성곱 신경망 모델을 많이 사용함 / 시계열 데이터: 시간적 관계를 잘 표현 → **시퀀셜** 편향을 가진 순환 신경망 모델을 많이 사용함
 - **시퀀셜**: 이전 시간의 상태를 기억하고 현재 입력과 결합해 다음 상태 예측 → 순차적 특징 강조

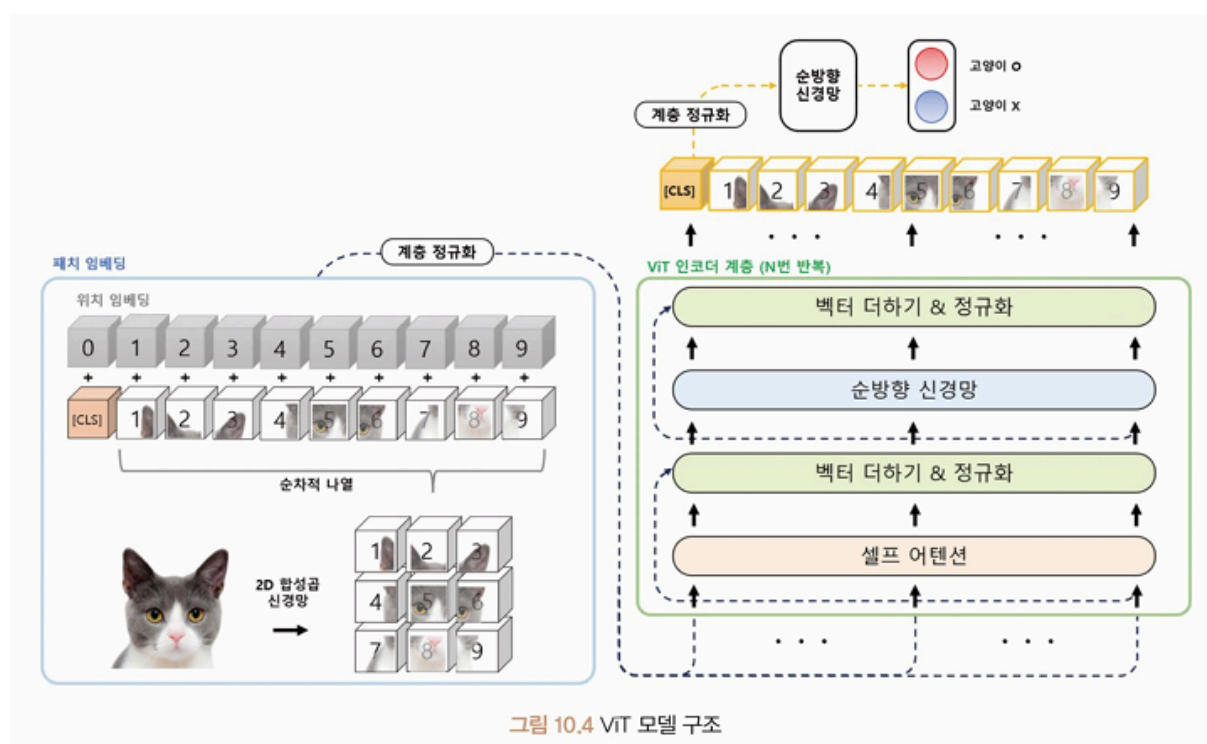
- ViT 모델과 귀납적 편향: 입력 데이터의 다양한 쿼리, 키, 값의 임베딩 형태로 **일반화된 관계 학습** → 귀납적 편향 거의 X, 대용량 데이터에 대해 이미지 특징들의 관계를 잘 학습하는 모델

표 10.1 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

ViT 모델

구조: Patch Embedding(입력 이미지→패치→벡터) + Encoder(각 패치 간 관계 학습)



패치 임베딩(Patch Embedding)

입력 이미지를 작은 패치로 분할하는 과정

패치 임베딩 과정

- 입력 이미지 640x480 → 224x224 정방형 크기로 이미지 크기 조절하는 전처리
- 이미지 패치 생성: 전체 이미지를 패치 크기로 분할해 시퀀셜 배열 생성
 - 하이퍼파라미터: 커널=패치 크기, 간격(stride)=패치 이동 폭

수식 10.1 패치 계산 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9 - 3}{3} + 1 = 3$$

- 분할된 이미지 패치들은 왼쪽→오른쪽, 위→아래 순서로 배열됨. 이 배열 가장 왼쪽에 **분류 토큰(Special Classification Token, CLS)** 추가. CLS는 전체 이미지를 대표하는 벡터로 특정 문제를 예측하는 데 사용됨.
 - 위치 임베딩(Position Embedding): 패치 위치 정보를 임베딩 벡터로 변환한 뒤 기존 이미지 패치 벡터와 더해서 인접 패치 간 관계 학습
 - 계층 정규화 적용
- 패치 임베딩을 통해 GPU 메모리 한계 극복, 더 큰 이미지 처리 가능

인코더 계층

- 벡터로 변환된 이미지 패치를 입력받아 다양한 쿼리, 키, 값 임베딩의 관계를 학습
- N개 인코더 레이어 반복적 적용
- 마지막 레이어 - 분류 토큰 벡터 추출 (이미지 데이터를 잘 표현하는 특징 벡터 → 이후 다양한 이미지 분류 및 검색 문제를 해결하는데 사용)
 - ex) 고양이인지 아닌지 구분하는 모델을 학습할 때 ViT 모델은 **Sequential Output**(마지막 ViT 인코더 계층에서 산출되는 이미지 패치들의 특징 벡터) 을 모두 사용하지 않고 전체 이미지의 특징을 잘 표현하는 **분류 토큰 벡터**만 사용해 분류 문제 해결

(624p 실습)

Swin Transformer (2021)

- 기존 트랜스포머 기반 모델 문제점
 - 고정된 패치 크기로 인해 세밀한 예측을 필요로 하는 semantic segmentation 작업에 적합하지 않았음
 - 트랜스포머의 셀프 어텐션은 입력 이미지 크기에 대한 2차(Quadratic) 계산 복잡도를 가지므로 고해상도 이미지를 처리하기 어려움
- 스윈 트랜스포머
 - 계층적 특징 맵을 구성해 1차(Linear) 계산 복잡도를 가지며, 트랜스포머 구조보다 이미지 특성을 더 잘 표현함

- Shifted Window 기술 이용: 입력 이미지를 일정한 크기의 패치로 분할하고 쌓아 window 구성함. 구성된 윈도우 영역만 셀프 어텐션 계산
 - 패치를 겹쳐 효율성 증가
 - 계층마다 어텐션이 수행되는 패치의 크기와 개수를 계층적으로 적용해 처리
→ 높은 해상도와 다양한 크기를 가진 객체를 효율적으로 처리할 수 있음
- 객체 탐지, 객체 분할 등의 작업에서 backbone 모델로 사용됨

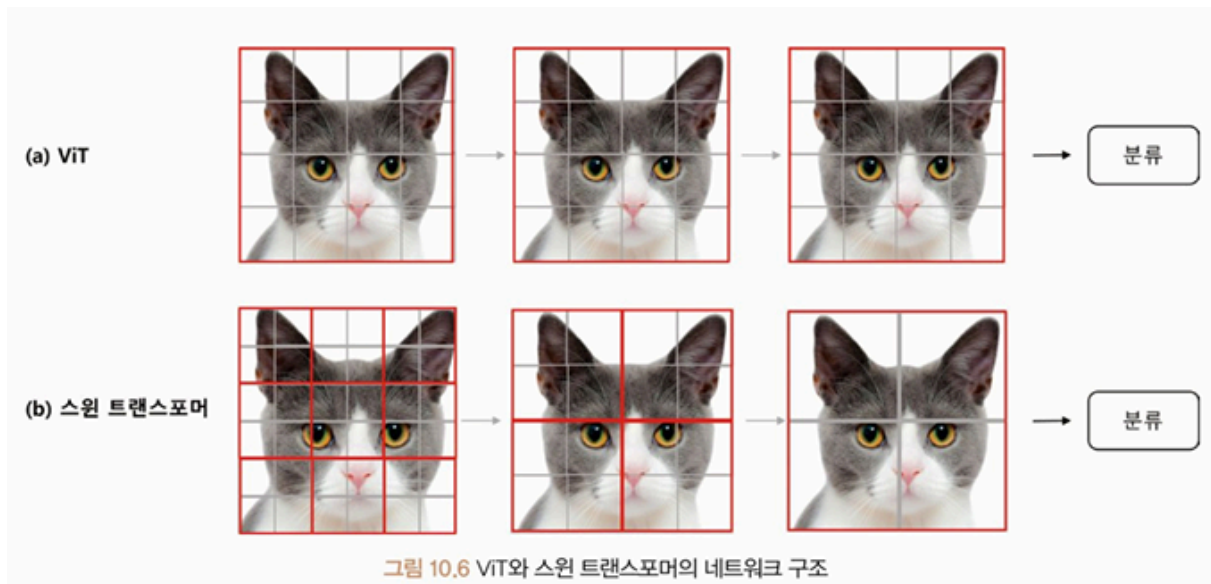
ViT와 스윈 트랜스포머 비교

• ViT

- **획일적인 패치 크기**, 위치에 대한 제약 → 다양한 이미지 크기와 종횡비를 가진 데이터 세트에 대해서 적용 어려움
 - 고양이 사진에서 계층마다 전체 이미지 안에 어텐션이 수행되는 패치 크기와 개수 동일
- 패치 단위 입력 → 이미지의 공간 정보가 완전히 보존되지 않을 수 있음
- 이미지 패치 수가 증가하면 학습 데이터의 크기가 커지고 계산량 증가 → 대규모 데이터셋에서 컴퓨팅 자원을 많이 사용하게 됨

• 스윈 트랜스포머

- local window를 활용해 물체의 크기나 해상도를 **계층적으로** 학습
 - 고양이 사진에서 계층마다 어텐션이 되는 패치의 크기와 개수를 계층적으로 다양하게 적용
- shift window 기술: 로컬 윈도우를 이동시키면서 입력 이미지를 일정한 크기의 패치로 분할하고, 특징 맵을 이동시켜 다음 계층에 사용함으로써 패치의 위치를 더 유연하게 다룸
- ViT보다 정확도가 더 높고 매개변수 수는 더 적음 → 더 넓은 범위의 이미지 크기와 종횡비를 다룰 수 O



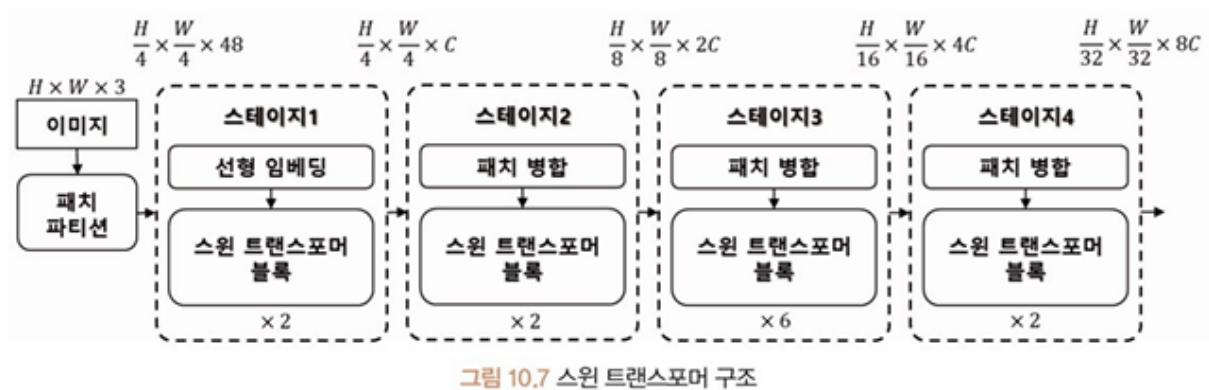
Swin transformer의 계층적 구조가 보임

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

스윈 트랜스포머 모델 구조

패치 파티션으로 구분된 이미지 → 선형 임베딩(Linear Embedding) → 스윈 트랜스포머 블록(Swin Transformer Block) → 패치 병합(Patch Merging) 반복



- 네 개의 스테이지: 각 스테이지는 서로 다른 해상도와 패치 크기를 가지며, 선형 임베딩-스윈 트랜스포머 블록-패치 병합 등 반복 적용
 - stage 1: 패치들이 선형 임베딩된 뒤 스윈 트랜스포머 블록에 전달
 - stage 2,3,4: 패치 병합 후 각 스윈 트랜스포머 블록에 전달
 - 패치 병합: 이미지 텐서 $[C, H, W] \rightarrow [2C, H/2, W/2]$ 재정렬, 저차원 임베딩 수행

패치 파티션 / 패치 병합

패치 파티션(Patch Partition): 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식

- 분할된 패치 → 트랜스포머 계층 입력
- 입력 이미지의 공간 정보를 보존하고 트랜스포머 계층에 대한 계산 효율성을 높임

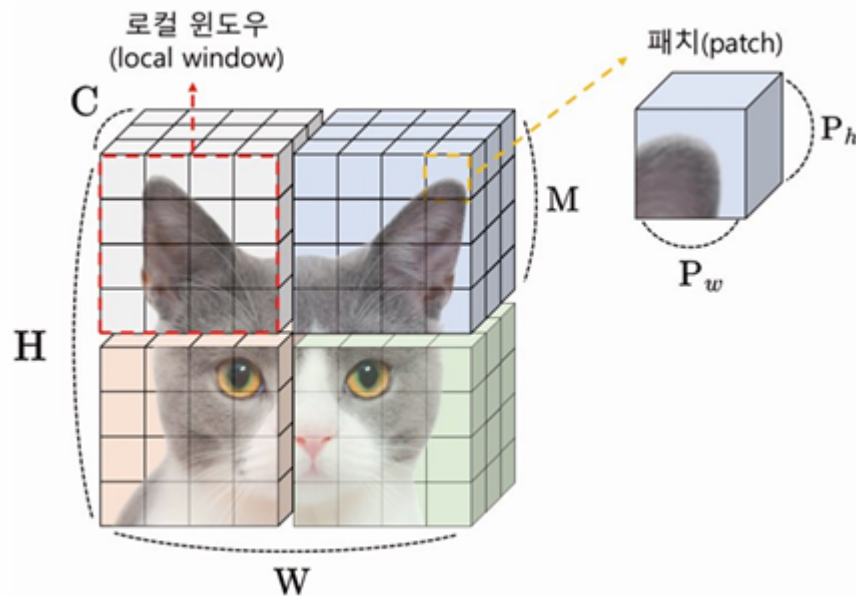


그림 10.8 패치 파티션 구조

- M: 로컬 윈도우 크기

패치 병합(Patch Merging): 인접한 패치들의 정보를 저차원으로 축소하는 과정

- $[C, H, W] \rightarrow [2C, H/2, W/2]$ 재정렬한 뒤 2C의 차원을 저차원으로 임베딩
- ex) 채널 3개, 이미지 패치 64개로 구성된 텐서 $[3, 8, 8]$
 - $[6, 4, 4]$ 재정렬
 - 증가된 6차원 채널을 3차원으로 임베딩: $[3, 4, 4]$ 텐서 생성
 - ⇒ 파라미터 수가 줄어듦

스원 트랜스포머 블록

- 패치 파티션 이후 4개 스테이지 안에서 반복 적용됨
- 선형 임베딩 또는 패치 병합 이후에 수행

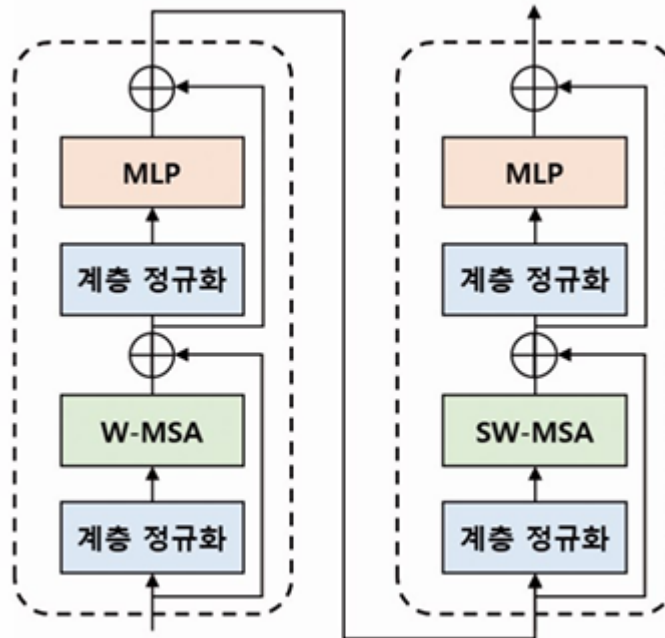


그림 10.9 스원 트랜스포머 블록 구조

- W-MSA: 로컬 윈도우를 사용하는 Multi-head Self Attention
 - 입력 특징 맵을 중첩되지 않게 나눈 뒤 각 윈도우에서 독립적 셀프 어텐션 수행
 - 이미지 내부 전체적인 어텐션 수행
 - ex) local window 빨간색 영역 안에서 패치 간 셀프 어텐션 수행
 - 특정 위치에 대한 정보 추출 → 각 윈도우 내의 지역적 특징 분석 가능
 - 전체 입력에 대한 셀프 어텐션 비용 2차 → 1차 계산 복잡도로 감소
- ** 연산량 대폭 감소 / but 윈도우 내부 이미지 패치 영역에만 셀프 어텐션을 수행한다는 단점
- SW-MSA: 윈도우 사이의 연결성을 구축해 윈도우 간의 관계성 정보 추출
 - 윈도우를 가로 또는 세로 방향으로 이동 → 인접한 윈도우 간의 셀프 어텐션 계산

W-MSA, SW-MSA의 연산량

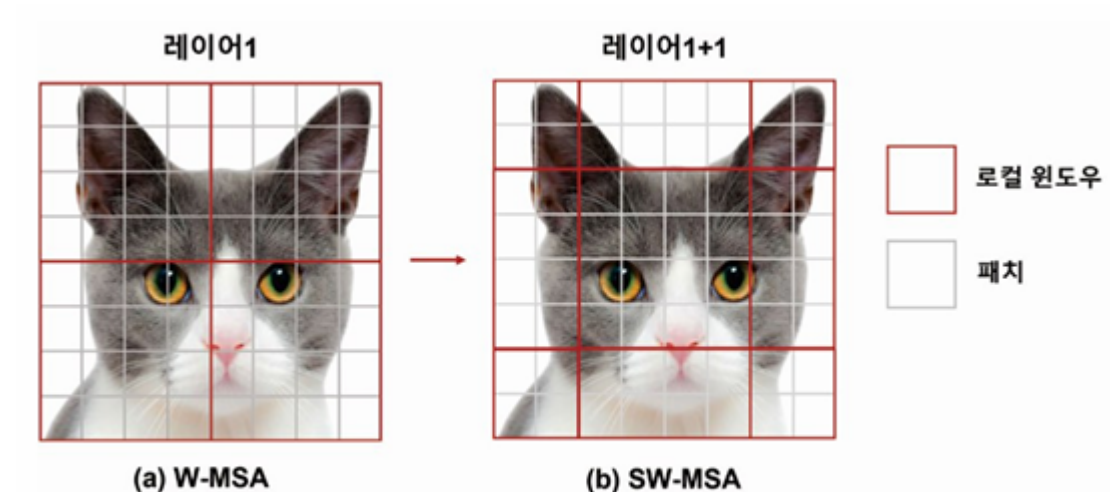


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

- W-MSA
 - **효율적인 배치 계산(Efficient Batch Computation)** -> 로컬 윈도우에서 연산을 수행하는 과정에서도 배치 축 사용 가능, 연산 속도 증가
 - 자연어 처리 작업에서 높은 성능을 보임
- SW-MSA
 - local window 간의 셀프 어텐션 수행 → 로컬 윈도우 개수 많아짐, 더 많은 셀프 어텐션 연산 요구
 - **순환 시프트(Cyclic Shift) + 어텐션 마스크(Attention Mask)** 사용
 - 순환 시프트: 로컬 윈도우가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져옴. 로컬 윈도우들을 $M/2$ 만큼 이동 (M : 윈도우 사이즈) 함.
 - + **어텐션 마스크**: 로컬 윈도우가 이동한 위치에서 연산을 수행하지 않게 함
 - 순환 시프트 단계에 어텐션 마스크를 추가해 셀프 어텐션 수행
 - 역순환 시프트(Reverse Cyclic Shift) 사용해 로컬 윈도우 구조로 복원
 - 효율적인 로컬 윈도우 간 셀프 어텐션 연산
 - 예를 들어

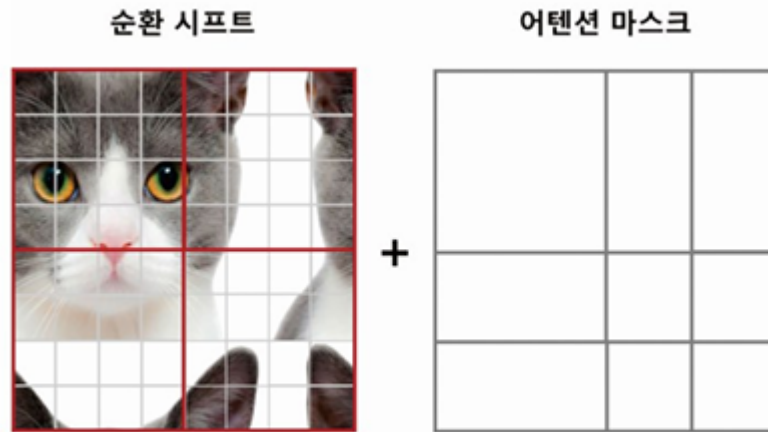


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

- 원래 총 9개의 윈도 파티션에 대해 9번의 셀프 어텐션 값을 구해야 함
- 4개의 윈도 파티션에 대한 4개의 셀프 어텐션 값만으로 모든 로컬 윈도 간의 셀프 어텐션 값 계산 가능

상대적 위치 편향(Relative Position Bias)

로컬 윈도 안에 있는 패치 간의 상대적인 거리 임베딩

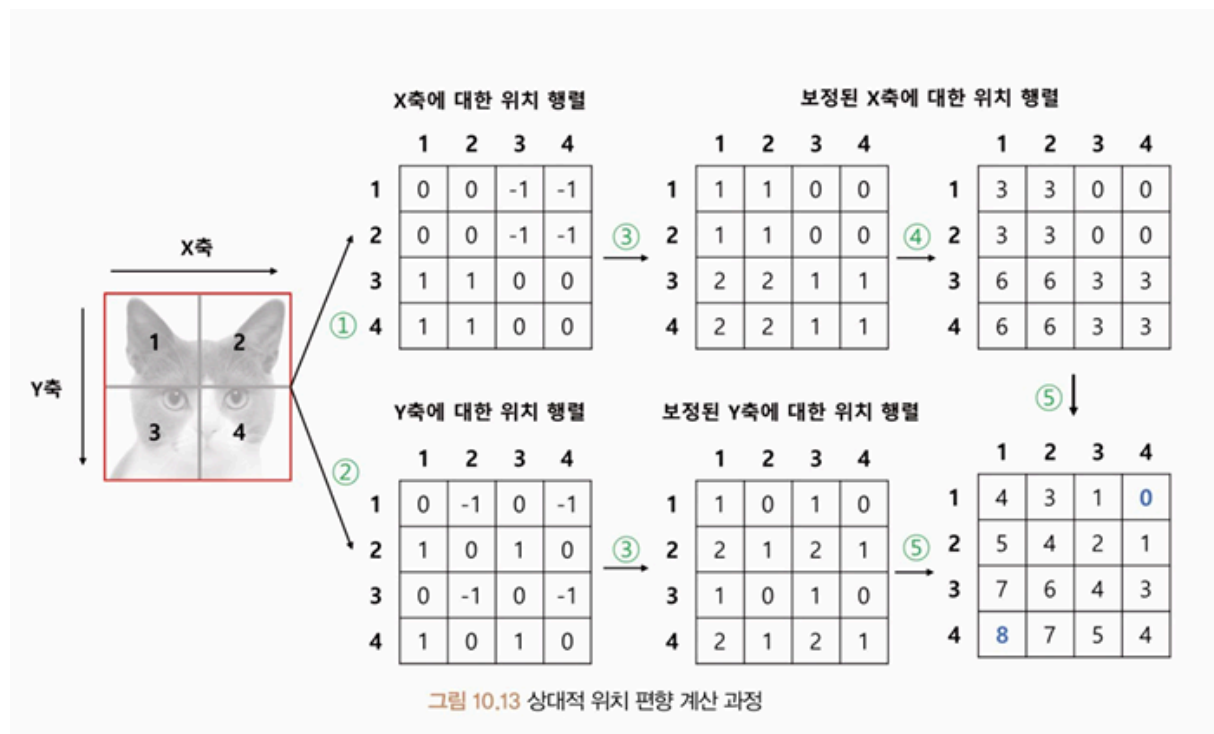


그림 10.13 상대적 위치 편향 계산 과정

- 이미지 패치들이 1~4로 주어짐
- 패치 간의 상대적인 {가로, 세로} 거리를 {x축, y축}으로 표기

- 코드적 해석

```
import torch

window_size = 2 # 여러 개의 이미지 패치를 포함하는 격자 크기. 윈도우 내외부
패치 구분
coords_h = torch.arange(window_size)
coords_w = torch.arange(window_size)
coords = torch.stack(torch.meshgrid([coords_h, coords_w], indexing="ij")) # 격자 생성
## torch.meshgrid → 사각형 x좌표, y좌표 배열 반환
## torch.stack → (x, y) 쌍 배열 생성
coords_flatten = torch.flatten(coords, 1)
relative_coords = coords_flatten[:, :, None] - coords_flatten[:, None, :]

print(relative_coords) # 1, 2 연산 과정
print(relative_coords.shape)
```

출력 결과

```
tensor([[[[ 0,  0, -1, -1],
          [ 0,  0, -1, -1],
          [ 1,  1,  0,  0],
          [ 1,  1,  0,  0]],

        [[ 0, -1,  0, -1],
          [ 1,  0,  1,  0],
          [ 0, -1,  0, -1],
          [ 1,  0,  1,  0]]]])
torch.Size([2, 4, 4])
```

```
# x, y축에 대한 위치 행렬
x_coords = relative_coords[0, :, :]
y_coords = relative_coords[1, :, :]

x_coords += window_size - 1 # X축에 대한 3번 연산 과정
```

```

y_coords += window_size - 1 # Y축에 대한 3번 연산 과정
x_coords *= 2 * window_size - 1 # 4번 연산 과정
print(f"X축에 대한 행렬:\n{x_coords}\n")
print(f"Y축에 대한 행렬:\n{y_coords}\n")

relative_position_index = x_coords + y_coords # 5번 연산 과정
print(f"X, Y축에 대한 위치 행렬:\n{relative_position_index}")

```

```

# x, y축에 대한 상대적 위치 좌표 반환
num_heads = 1 # MSA 계산 시 사용한 헤드 수
relative_position_bias_table = torch.Tensor( # 헤드별 0~8번 9개의 상대적
위치 편향을 가지는 테이블
    torch.zeros((2 * window_size - 1) * (2 * window_size - 1), num_head
s)
)

relative_position_bias = relative_position_bias_table[relative_position_in
dex.view(-1)]
# 해당 위치 편향 값 불러옴
relative_position_bias = relative_position_bias.view(
    window_size * window_size, window_size * window_size, -1
)
# 헤드별 i번째 패치에서 j번째 패치의 어텐션 값에 더해지는 텐서 [4,4,1]로 재구
성
print(relative_position_bias.shape)

## 출력: torch.Size([1,4,4])

```

→ 상대적 위치 임베딩(B) 생성됨

⇒ 셀프 어텐션 수식에 적용

수식 10.2 상대적 위치 편향을 고려한 셀프 어텐션

$$Attention(Q, K, V) = SoftMax(QK^T / \sqrt{d} + B)V$$

(실습 651~)

CvT (Convolutional Vision Transformer)

합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델

ViT & CvT

**** ViT의 셀프 어텐션:** 이미지 정보를 바탕으로 각 패치 간의 관계를 학습하지만, 모든 패치에 대해 동○리한 크기와 간격을 사용하므로 이미지의 물체 크기나 해상도를 계층적으로 학습하기 어려움 (경계에 어떤 물체의 핵심적인 부분이 위치할 수 있음)

- 스윈 트랜스포머: 트랜스포머 블록마다 다른 로컬 윈도우 크기를 사용해 이미지의 계층적 특징 학습
- **CvT:** 합성곱 신경망에서 사용하는 계층적 구조를 ViT에 적용 → 이미지의 지역 특징(Local Feature)과 전역 특징(Global Feature)을 모두 활용해 비교적 적은 수의 매개변수로 높은 성능을 보임
 - local feature ← 합성곱 계층에서 추출
 - global feature ← 셀프 어텐션 계층에서 결합
 - 얼굴 이미지가 CvT 모델의 입력으로 사용될 때
 - 저수준 계층: 선 또는 점에 대한 특징
 - 중간 수준: 눈이나 귀에 대한 특징
 - 고수준: 얼굴 구조에 대한 특징

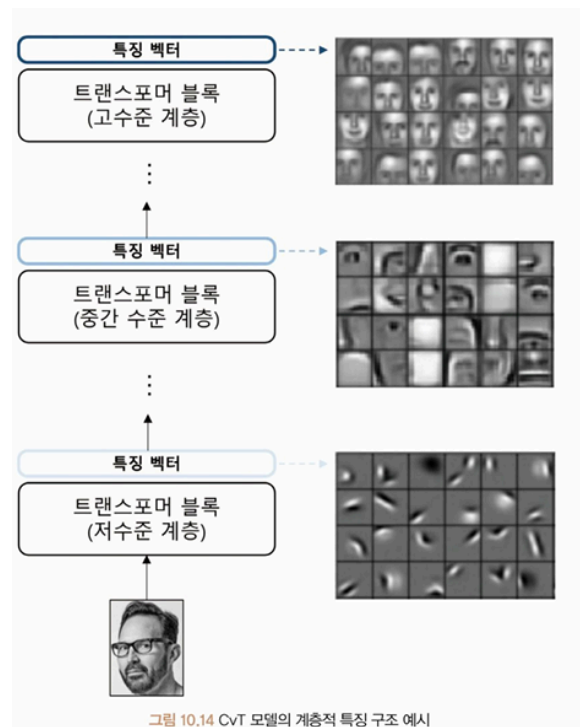


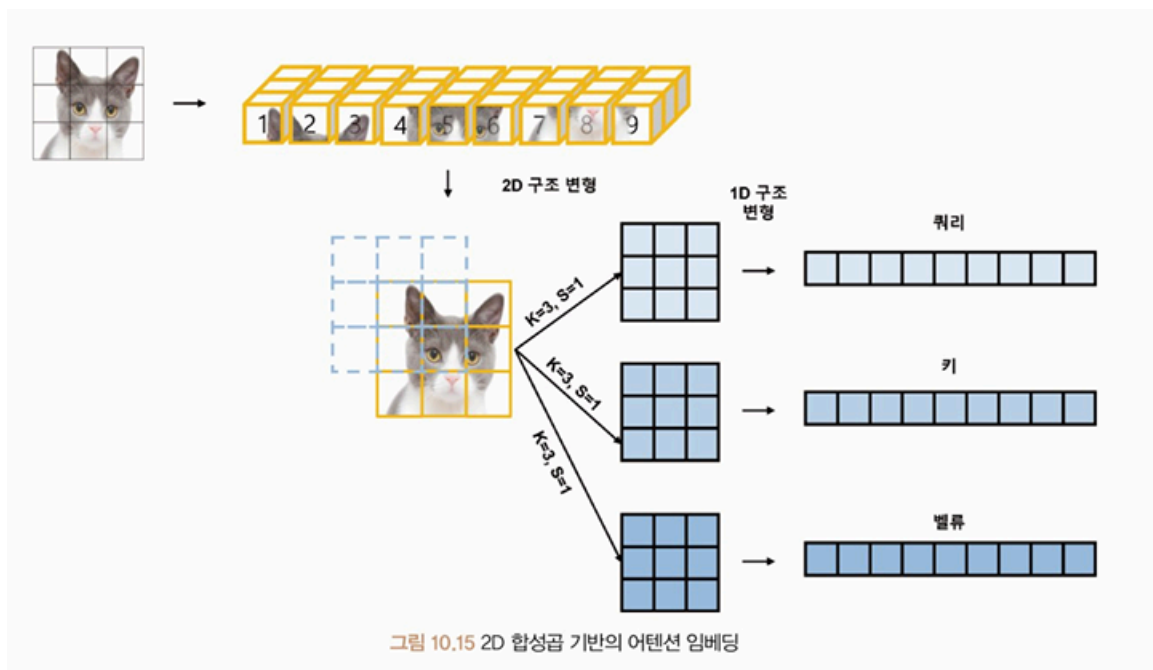
표 10.4 ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

합성곱 토큰 임베딩 (Conv Token Embedding)

2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩으로, ViT에 계층적인 합성곱 신경망의 성질을 추가하기 위해 사용됨

- 기존 트랜스포머 모델: 9개의 이미지 패치 사용, 선형 임베딩을 통해 쿼리, 키, 값 임베딩으로 변환
 → 임베딩을 사용해 셀프 어텐션을 계산하고 출력을 생성
 ⇒ 이미지의 근본적인 2D 모양 구조를 잃음
- CvT 모델: 이미지의 2D 구조를 보존하면서도 셀프 어텐션을 계산하는 방식
 - 이미지 패치 — 2D Conv Embedding → 쿼리, 키, 값 임베딩으로 변환 ⇒ 셀프 어텐션 계산 & 출력 생성



- 이미지에 3x3 kernel, 1 stride 적용 → 3x3 패치 생성
 → 각 패치를 쿼리, 키, 값에 대해 합성곱 연산 적용
 → 1차원 구조로 재배열
 ⇒ 기존 셀프 어텐션 방법으로 학습

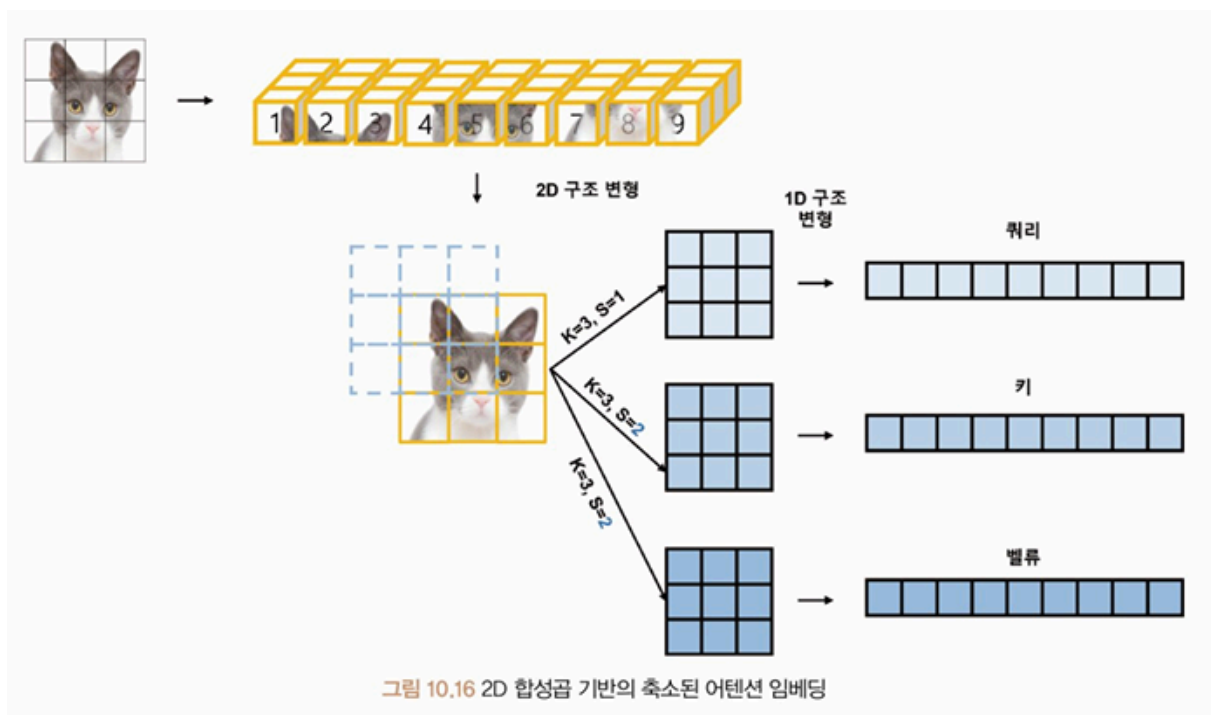
⇒ 지역 특징 학습 가능, 시퀀스 길이를 줄이는 동시에 토큰 특징의 차원을 증가시킴. 합성곱 모델에서 수행되는 방식처럼 다운 샘플링과 특징 맵의 수를 늘리게 됨

어텐션에 대한 합성곱 임베딩 (Conv Projection for Attention)

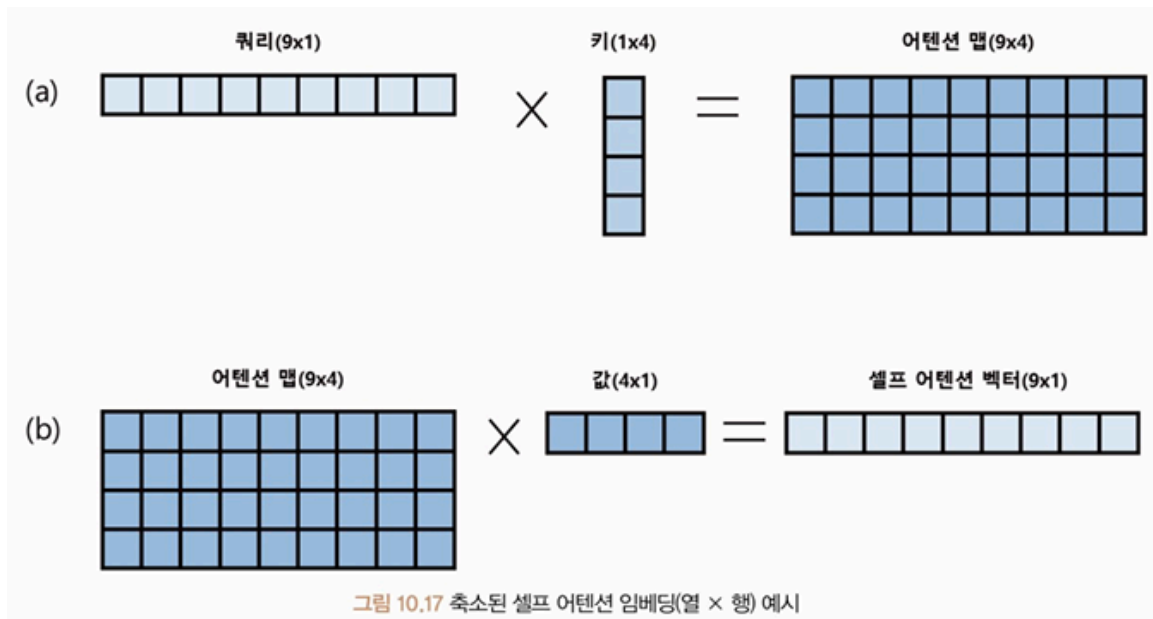
2D로 재구성된 토큰 맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산

- 매개변수의 수를 줄이기 위해 쿼리, 키, 값의 차원을 축소시킨 것

Squeezed Convolutional Projection



- 쿼리: 3x3 kernel, 1 stride 적용 → 9개 이미지 패치 생성
키, 값: 3x3 kernel, 2 stride 적용 → 4개 이미지 패치 생성
- 쿼리(9), 키, 값(4)의 길이가 달라졌지만 셀프 어텐션의 결괏값 사이즈가 9x1로 동일하게 출력될 수 있음



- **어텐션 맵** → 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 **가중치** 반영
- 입력 벡터(쿼리 벡터), 출력 벡터(셀프 어텐션 벡터)의 벡터의 패치 길이가 동일함
→ 계산해야 되는 이미지 패치 개수가 줄어들어 연산량이 감소함

CvT 모델은 트랜스포머 계층이 깊어질수록, 패치 간 관계를 학습하는 길이를 축소시키는 대신 벡터의 차원을 증가시켜 모델의 매개변수 수를 대폭 감소시킴

- ex)
 - 첫 번째 블록 이미지 패치 크기 64×64 , 임베딩 차원 128
 - 마지막 블록 이미지 패치 크기 8×8 ($\times 1/4$), 임베딩 차원 512 ($\times 4$)
- 네트워크의 안정성 향상, 모델 설계 단순화 효과

(실습 665~)